

SYSEN 5888 Spring 2026

Jonathan Lloyd

Homework 2, Question 2

Goal: ConvNets while renowned for their prowess in image processing, have also demonstrated strong capabilities in handling sequential data such as text. In this problem, you will be applying these principles of CNNs to a classic problem in natural language processing - sentiment analysis.

Tools: Numpy, PyTorch, Keras (TensorFlow)

Data: IMDB movie reviews dataset provided by `tensorflow.keras.datasets.imdb`

Task: Load the IMDB dataset from Keras using a vocabulary size of 2000 for tokenization & numericalization. Each review in the dataset is already pre-processed and encoded as a sequence of word indexes. A mapping between words and their corresponding indexes is provided using the `imdb.get_word_index()` method. For consistent input to the model, your task is to pad the reviews or truncate them to a uniform length. This can be achieved using the `pad_sequences` method from Keras to convert all reviews to a length of 300 words using the `maxlen` argument in the `pad_sequences` method. The preprocessed NumPy arrays will then be fed into a PyTorch CNN for sentiment classification.

```
In [2]: # Update packages in Colab server to latest stable versions
%pip install --upgrade torch tensorflow numpy pandas matplotlib
```

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.4.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (3.0.1)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.8)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.24.3)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)

Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.3.5)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)

Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)

Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)

Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.1)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.78.1)

Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)

Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=3 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.46.3)

Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)

Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.0)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2026.1.4)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.20.0->tensorflow) (3.10.2)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.20.0->tensorflow) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.20.0->tensorflow) (3.1.6)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (4.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (2.19.2)

Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.2)

```
In [3]: # Import necessary Libraries
import os
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import tensorflow as tf
import pandas as pd
from torch.utils.data import DataLoader, TensorDataset
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Hyperparameters for data processing
VOCAB_SIZE = 2000 # per homework
MAX_LEN = 300 # fixed sequence length
BATCH_SIZE = 32 # per homework
SHUFFLE_SEED = 42

# Load IMDB dataset from Keras (already tokenized and indexed)
(X_train_full, y_train_full), (X_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)

# Get the word -> index mapping (for analysis / interpretability)
word_index = imdb.get_word_index()
```

```

# Pad and truncate to fixed length of 300 tokens
X_train_full = pad_sequences(X_train_full, maxlen=MAX_LEN, padding="post", truncating="post")
X_test = pad_sequences(X_test, maxlen=MAX_LEN, padding="post", truncating="post")

# Create validation split from training data
np.random.seed(SHUFFLE_SEED)
indices = np.random.permutation(len(X_train_full))
X_train_full = X_train_full[indices]
y_train_full = np.array(y_train_full)[indices]

n_val = 1000 # per homework
X_val = X_train_full[:n_val]
y_val = y_train_full[:n_val]
X_train = X_train_full[n_val:]
y_train = y_train_full[n_val:]

# Summary
print(f"Training samples: {len(X_train)}, Validation samples: {len(X_val)}, Test samples: {len(X_test)}")
print(f"Sequence length: {MAX_LEN}, Batch size: {BATCH_SIZE}, Vocab size (num_words): {num_words}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>

17464789/17464789 ————— 1s 0us/step

/usr/local/lib/python3.12/dist-packages/numpy/lib/_format_impl.py:838: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

```
array = pickle.load(fp, **pickle_kwargs)
```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json

1641221/1641221 ————— 1s 1us/step

Training samples: 24000, Validation samples: 1000, Test samples: 25000

Sequence length: 300, Batch size: 32, Vocab size (num_words): 2000

Architecture:

The architecture of the convolutional neural network model for this problem is as follows:

Embedding Layer: Input Vocabulary Size: 2000 words Embedding Dimension: 16 Input Length: 300 words

Conv1D Layer: Filters: 128 Kernel Size: 3 Activation: ReLU Stride: 1 Padding: Valid

GlobalMaxPooling1D Layer

Dense Layer: Units: 1 Activation: Sigmoid

```

In [4]: # PyTorch CNN Model Definition
class TextCNN(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        embed_dim: int = 16,
        num_filters: int = 128,
        kernel_size: int = 3,

```

```

) -> None:
    super().__init__()
    # Embedding layer: maps word indices to dense vectors
    self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

    # 1D convolution over the sequence (time) dimension
    self.conv = nn.Conv1d(
        in_channels=embed_dim,
        out_channels=num_filters,
        kernel_size=kernel_size,
    )

    # Global max pooling over the time dimension
    self.global_max_pool = nn.AdaptiveMaxPool1d(output_size=1)

    # Final classification layer
    self.fc = nn.Linear(num_filters, 1)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """Forward pass.

    x: LongTensor of shape (batch_size, seq_len)
    returns: probabilities of shape (batch_size,)
    """
    # (batch, seq_len) -> (batch, seq_len, embed_dim)
    embedded = self.embedding(x)
    # (batch, seq_len, embed_dim) -> (batch, embed_dim, seq_len)
    embedded = embedded.permute(0, 2, 1)

    # Convolution + ReLU
    conv_out = torch.relu(self.conv(embedded)) # (batch, num_filters, L')

    # Global max pooling over time -> (batch, num_filters, 1)
    pooled = self.global_max_pool(conv_out).squeeze(-1) # (batch, num_filters)

    # Linear layer to a single logit
    logits = self.fc(pooled).squeeze(-1) # (batch,)

    # Sigmoid for binary sentiment probability
    probs = torch.sigmoid(logits)
    return probs

# Instantiate model (will be moved to appropriate device in training cell)
EMBED_DIM = 16
NUM_FILTERS = 128
KERNEL_SIZE = 3

# VOCAB_SIZE is defined in the preprocessing cell
model = TextCNN(vocab_size=VOCAB_SIZE, embed_dim=EMBED_DIM, num_filters=NUM_FILTERS)
print(model)

```

```

TextCNN(
    (embedding): Embedding(2000, 16, padding_idx=0)
    (conv): Conv1d(16, 128, kernel_size=(3,), stride=(1,))
    (global_max_pool): AdaptiveMaxPool1d(output_size=1)
    (fc): Linear(in_features=128, out_features=1, bias=True)
)

```

Training: The model should be compiled using the 'binary_crossentropy' as the loss function and 'adam' optimizer. Additionally, 'accuracy' should be assigned as the main metric. A subset of the training data (1000 samples) should be set aside as a validation set, while the rest should be used for training. The model should be trained for a total of 30 (or 10) epochs, with a batch size of 32. After training, the model should be evaluated on the test data to obtain the final accuracy score. This will give a measure of how well the model can generalize to unseen reviews.

```

In [5]: ## Helper Functions

# BASIC TRAIN / EPOCH
# Mirroring HW2_Q1 structure but adapted for binary sentiment labels.
def train(model, loader, criterion, optimizer, device):
    """Train the model for one epoch."""
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in loader:
        inputs = inputs.to(device)
        # BCE Loss expects floating labels in {0, 1}
        labels = labels.float().to(device)

        optimizer.zero_grad()
        outputs = model(inputs) # (batch,)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)

        # Convert probabilities to class predictions using 0.5 threshold
        preds = (outputs >= 0.5).float()
        total += labels.size(0)
        correct += (preds == labels).sum().item()

    epoch_loss = running_loss / len(loader.dataset)
    epoch_acc = correct / total
    return epoch_loss, epoch_acc

# BASIC EVAL / EPOCH
def evaluate(model, loader, criterion, device):
    """Evaluate the model on validation or test data."""
    model.eval()
    running_loss = 0.0

```

```

correct = 0
total = 0

with torch.no_grad():
    for inputs, labels in loader:
        inputs = inputs.to(device)
        labels = labels.float().to(device)

        outputs = model(inputs)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * inputs.size(0)

        preds = (outputs >= 0.5).float()
        total += labels.size(0)
        correct += (preds == labels).sum().item()

epoch_loss = running_loss / len(loader.dataset)
epoch_acc = correct / total
return epoch_loss, epoch_acc

# PLOT FUNCTIONS (SAVE TO DISK LIKE HW2_Q1)

def plot_loss(curve, dataset, output_dir="Plot JPGs"):
    """Plot and save a single loss curve (training or validation)."""
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    plt.figure()
    plt.plot(curve)
    plt.xlabel("Epoch")
    plt.ylabel(f"{dataset} Loss")
    plt.title(f"Binary Cross-Entropy Loss - {dataset}")
    filename = os.path.join(output_dir, f"LossCurve_{dataset}.jpg")
    plt.savefig(filename)
    plt.close()

def plot_accuracy(curve, dataset, output_dir="Plot JPGs"):
    """Plot and save a single accuracy curve (training or validation)."""
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    plt.figure()
    plt.plot(curve)
    plt.xlabel("Epoch")
    plt.ylabel(f"{dataset} Accuracy")
    plt.title(f"Accuracy - {dataset}")
    filename = os.path.join(output_dir, f"AccuracyCurve_{dataset}.jpg")
    plt.savefig(filename)
    plt.close()

```

```

In [6]: # MODEL TRAIN - ALL EPOCHS
# Follows HW2_Q1-style training loop.
def train_model(model, train_loader, val_loader, criterion, optimizer, device, num_
    """Train the model and evaluate on validation data after each epoch."""

```



```

training_loss_curve = []
validation_loss_curve = []
training_accuracy = []
validation_accuracy = []

for epoch in range(num_epochs):
    train_loss, train_acc = train(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)

    training_loss_curve.append(train_loss)
    training_accuracy.append(train_acc)
    validation_loss_curve.append(val_loss)
    validation_accuracy.append(val_acc)

    print(
        f"Epoch {epoch + 1}/{num_epochs} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
    )

return training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy

```

```

In [7]: # MAIN EXPERIMENT HELPER
# Uses HW2_Q1-style run_experiment pattern adapted to IMDB sentiment.
def run_experiment(max_epochs=30, batch_size=BATCH_SIZE):
    """Run full experiment: build loaders, train model, evaluate on test set."""
    # Detect device (GPU on Colab if available)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # Build PyTorch datasets and Loaders from preprocessed NumPy arrays
    train_dataset = TensorDataset(
        torch.from_numpy(X_train).long(),
        torch.from_numpy(y_train).float(),
    )
    val_dataset = TensorDataset(
        torch.from_numpy(X_val).long(),
        torch.from_numpy(y_val).float(),
    )
    test_dataset = TensorDataset(
        torch.from_numpy(X_test).long(),
        torch.from_numpy(y_test).float(),
    )

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    # Instantiate model fresh for this run and move to device
    model = TextCNN(
        vocab_size=VOCAB_SIZE,
        embed_dim=EMBED_DIM,
        num_filters=NUM_FILTERS,
        kernel_size=KERNEL_SIZE,
    )
    model.to(device)

```

```

# Binary cross-entropy loss and Adam optimizer
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters())

# Train
training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy = train(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    device,
    num_epochs=max_epochs,
)

# Final test evaluation
test_loss, test_acc = evaluate(model, test_loader, criterion, device)

# Save individual curves for later quad plotting
plot_loss(training_loss_curve, "Training")
plot_loss(validation_loss_curve, "Validation")
plot_accuracy(training_accuracy, "Training")
plot_accuracy(validation_accuracy, "Validation")

# Prepare summary row
row = {
    "Model Name": "TextCNN",
    "Epochs": max_epochs,
    "Training Accuracy (%)": training_accuracy[-1] * 100.0,
    "Validation Accuracy (%)": validation_accuracy[-1] * 100.0,
    "Test Accuracy (%)": test_acc * 100.0,
    "Final Test Loss": test_loss,
}

return row, {
    "train_loss": training_loss_curve,
    "val_loss": validation_loss_curve,
    "train_acc": training_accuracy,
    "val_acc": validation_accuracy,
}

```

```

In [8]: ## Train and Run Model
# binary_crossentropy loss, ADAM optimizer
# Run for 30 epochs, batch size 32
# Evaluate on test data to obtain accuracy score

# Define results table, maximum epochs to run
results_dataframe = pd.DataFrame(
    columns=[
        "Model Name",
        "Epochs",
        "Training Accuracy (%)",
        "Validation Accuracy (%)",
        "Test Accuracy (%)",
        "Final Test Loss",
    ]
)

```

```
    ]
)
MAX_EPOCHS = 30

# Run experiment
row, history = run_experiment(max_epochs=MAX_EPOCHS)
results_dataframe = pd.concat([results_dataframe, pd.DataFrame([row])], ignore_index=True)
print(results_dataframe)
```

Using device: cuda

Epoch 1/30	Train Loss: 0.6044	Train Acc: 0.6659	Val Loss: 0.5141	Val Acc: 0.7410
Epoch 2/30	Train Loss: 0.4444	Train Acc: 0.7945	Val Loss: 0.4215	Val Acc: 0.7980
Epoch 3/30	Train Loss: 0.3675	Train Acc: 0.8390	Val Loss: 0.4137	Val Acc: 0.8070
Epoch 4/30	Train Loss: 0.3213	Train Acc: 0.8615	Val Loss: 0.3651	Val Acc: 0.8400
Epoch 5/30	Train Loss: 0.2838	Train Acc: 0.8837	Val Loss: 0.3468	Val Acc: 0.8510
Epoch 6/30	Train Loss: 0.2526	Train Acc: 0.8990	Val Loss: 0.3794	Val Acc: 0.8360
Epoch 7/30	Train Loss: 0.2284	Train Acc: 0.9101	Val Loss: 0.3338	Val Acc: 0.8620
Epoch 8/30	Train Loss: 0.2056	Train Acc: 0.9231	Val Loss: 0.3300	Val Acc: 0.8650
Epoch 9/30	Train Loss: 0.1863	Train Acc: 0.9307	Val Loss: 0.3375	Val Acc: 0.8640
Epoch 10/30	Train Loss: 0.1678	Train Acc: 0.9387	Val Loss: 0.3457	Val Acc: 0.8630
Epoch 11/30	Train Loss: 0.1482	Train Acc: 0.9495	Val Loss: 0.3570	Val Acc: 0.8620
Epoch 12/30	Train Loss: 0.1326	Train Acc: 0.9554	Val Loss: 0.3547	Val Acc: 0.8650
Epoch 13/30	Train Loss: 0.1193	Train Acc: 0.9620	Val Loss: 0.3624	Val Acc: 0.8670
Epoch 14/30	Train Loss: 0.1037	Train Acc: 0.9690	Val Loss: 0.4023	Val Acc: 0.8520
Epoch 15/30	Train Loss: 0.0910	Train Acc: 0.9747	Val Loss: 0.3932	Val Acc: 0.8600
Epoch 16/30	Train Loss: 0.0781	Train Acc: 0.9812	Val Loss: 0.4122	Val Acc: 0.8590
Epoch 17/30	Train Loss: 0.0671	Train Acc: 0.9848	Val Loss: 0.4188	Val Acc: 0.8570
Epoch 18/30	Train Loss: 0.0552	Train Acc: 0.9905	Val Loss: 0.4353	Val Acc: 0.8500
Epoch 19/30	Train Loss: 0.0461	Train Acc: 0.9935	Val Loss: 0.4521	Val Acc: 0.8490
Epoch 20/30	Train Loss: 0.0378	Train Acc: 0.9958	Val Loss: 0.4644	Val Acc: 0.8540
Epoch 21/30	Train Loss: 0.0322	Train Acc: 0.9975	Val Loss: 0.4898	Val Acc: 0.8490
Epoch 22/30	Train Loss: 0.0256	Train Acc: 0.9984	Val Loss: 0.4983	Val Acc: 0.8520
Epoch 23/30	Train Loss: 0.0197	Train Acc: 0.9996	Val Loss: 0.5255	Val Acc: 0.8510
Epoch 24/30	Train Loss: 0.0165	Train Acc: 0.9997	Val Loss: 0.5418	Val Acc: 0.8470
Epoch 25/30	Train Loss: 0.0126	Train Acc: 0.9998	Val Loss: 0.5610	Val Acc: 0.8550
Epoch 26/30	Train Loss: 0.0099	Train Acc: 0.9999	Val Loss: 0.5955	Val Acc: 0.8500
Epoch 27/30	Train Loss: 0.0079	Train Acc: 0.9999	Val Loss: 0.6149	Val Acc: 0.8500
Epoch 28/30	Train Loss: 0.0063	Train Acc: 1.0000	Val Loss: 0.6362	Val Acc:

0.8480

Epoch 29/30 | Train Loss: 0.0048 | Train Acc: 1.0000 | Val Loss: 0.6521 | Val Acc: 0.8510

Epoch 30/30 | Train Loss: 0.0035 | Train Acc: 1.0000 | Val Loss: 0.7586 | Val Acc: 0.8500

	Model Name	Epochs	Training Accuracy (%)	Validation Accuracy (%)	\
0	TextCNN	30	100.0	85.0	

	Test Accuracy (%)	Final Test Loss
0	84.904	0.742207

Visualization: Plot the accuracy and loss for both training and validation datasets across epochs to analyze the performance of the model over epochs.

```
In [9]: # Plots across epochs
# Plot accuracy and loss for both training and validation - build a quad
# Uses history dict returned by run_experiment()

epochs = range(1, len(history["train_loss"]) + 1)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Top-left: Training Loss
axes[0, 0].plot(epochs, history["train_loss"], color="C0")
axes[0, 0].set_xlabel("Epoch")
axes[0, 0].set_ylabel("Loss")
axes[0, 0].set_title("Training Loss")
axes[0, 0].grid(True)

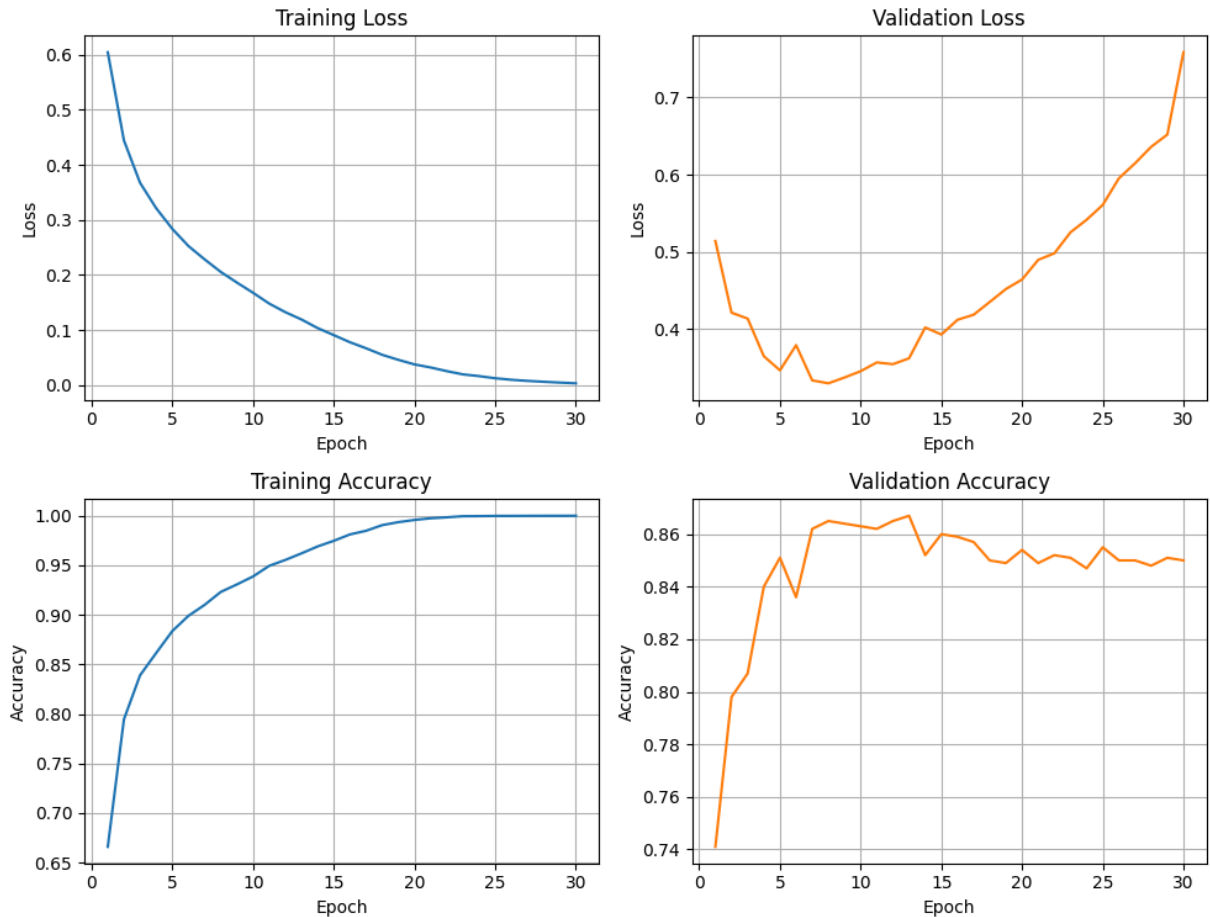
# Top-right: Validation Loss
axes[0, 1].plot(epochs, history["val_loss"], color="C1")
axes[0, 1].set_xlabel("Epoch")
axes[0, 1].set_ylabel("Loss")
axes[0, 1].set_title("Validation Loss")
axes[0, 1].grid(True)

# Bottom-left: Training Accuracy
axes[1, 0].plot(epochs, history["train_acc"], color="C0")
axes[1, 0].set_xlabel("Epoch")
axes[1, 0].set_ylabel("Accuracy")
axes[1, 0].set_title("Training Accuracy")
axes[1, 0].grid(True)

# Bottom-right: Validation Accuracy
axes[1, 1].plot(epochs, history["val_acc"], color="C1")
axes[1, 1].set_xlabel("Epoch")
axes[1, 1].set_ylabel("Accuracy")
axes[1, 1].set_title("Validation Accuracy")
axes[1, 1].grid(True)

plt.suptitle("IMDB TextCNN: Loss and Accuracy Across Epochs", fontsize=12, y=1.02)
plt.tight_layout()
plt.show()
```

IMDB TextCNN: Loss and Accuracy Across Epochs



```
In [10]: # Plot loss and accuracy on top of each other
epochs = range(1, len(history["train_loss"]) + 1)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

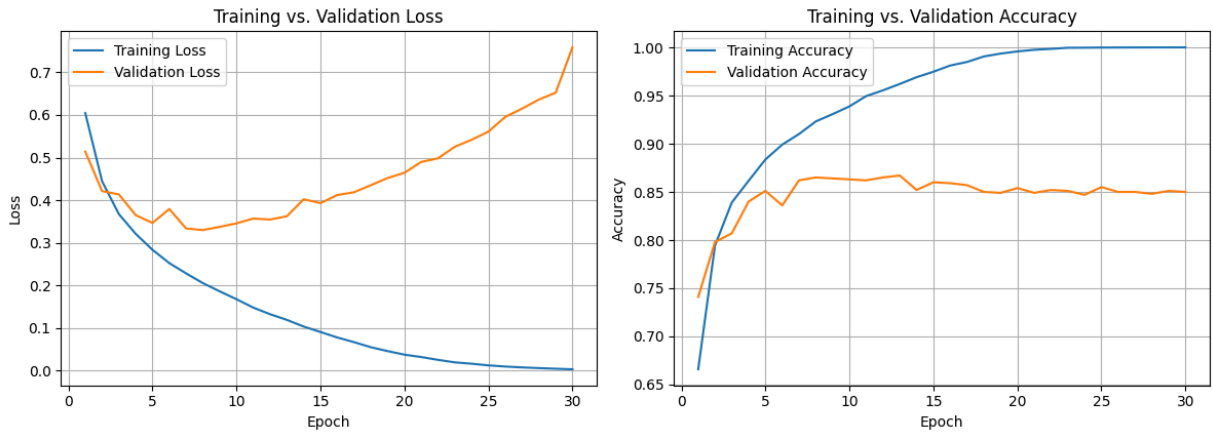
# Left subplot: Training and Validation Loss
axes[0].plot(epochs, history["train_loss"], label="Training Loss", color="C0")
axes[0].plot(epochs, history["val_loss"], label="Validation Loss", color="C1")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].set_title("Training vs. Validation Loss")
axes[0].legend()
axes[0].grid(True)

# Right subplot: Training and Validation Accuracy
axes[1].plot(epochs, history["train_acc"], label="Training Accuracy", color="C0")
axes[1].plot(epochs, history["val_acc"], label="Validation Accuracy", color="C1")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("Training vs. Validation Accuracy")
axes[1].legend()
axes[1].grid(True)

plt.suptitle("IMDB TextCNN: Loss and Accuracy Across Epochs", fontsize=14, y=1.05)
```

```
plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # Adjust layout to prevent supitle overl  
plt.show()
```

IMDB TextCNN: Loss and Accuracy Across Epochs



Deliverables:

1. Model Accuracy and Loss Curves: A detailed report of the performance of the model, focusing on accuracy and loss curves.
2. Analysis of Model Performance: A thorough analysis should be conducted to discuss the results obtained from the model. This analysis should include
 - 2a. Whether the model overfits or underfits the training data.
 - 2b. Examination of the loss and accuracy curves to identify potential indicators of the model's behavior (such as plateaus or sharp changes).

Performance Discussion

The loss curves show training loss decreasing smoothly to 0 over 30 epochs, the validation curve in a U-shape, the training accuracy increasing smoothly to plateau at 1, and validation accuracy hovering around 85%. Test accuracy as reported textually was just shy of that 85%.

These curves clearly show the model overfitting. The validation loss climbs after Epoch 7 and does not stop. This divergence is a key hallmark of overfit models. Methods to improve the model fit include adding an early stopping metric and increasing dropout.